

Unicesumar

Engenharia de Software

Paulo Ricardo Rinaldi da Silva

Trabalho de Linguagens e Técnicas de Programação

Atividade Prática – Métodos de ordenação em C

Maringá 2025

Sumário

1. Métodos de ordenação	
1.1 O que é um método de ordenação.....	
2. Método Bubble Sort	
2.1 O que é.....	
2.2 NíveldeComplexidade.....	
2.3 Como funciona.....	
2.4 Vantagens e Desvantagens.....	
2.5 Código exemplo.....	
3. Método Selection Sort	
3.1 O que é.....	
3.2 Para que serve.....	
3.3 Implementação.....	
3.4 Comparação.....	
4. Método Insertion Sort	
4.1 O que é.....	
4.2 Como funciona	
4.3 Vantagens e Desvantagens.....	
4.4 Comparações.....	
4.5 Código exemplo.....	
5. Método Merge Sort	
5.1 O que é.....	
5.2 Como funciona	

5.3 Vantagens e Desvantagens.....	
5.4 Comparações.....	
5.5 Código exemplo.....	
6. Método Quick Sort	
6.1 O que é.....	
6.2 Como funciona	
6.3 Vantagens e Desvantagens.....	
6.4 Comparações.....	
6.5 Código exemplo.....	
7. Referências.....	

1. O que é uma metodologia de ordenação

Uma metodologia de ordenação são algoritmos criados para organizar vetores e Arrays, por isso foram criadas diferentes metodologias a serem aplicadas em diferentes contextos com diferentes tipos de experiência e complexidade do código, são analisadas de forma geral pelo seu desempenho, eficiência, e velocidade.

2 Método Bubble Sort

2.1 O que é

O método de ordenação Bubble Sort foi descrito pela primeira vez em 1956 por Edward Harry Friend, sendo posteriormente denominado “Bubble Sort” por Kenneth E. Iverson.

Trata-se de um algoritmo de ordenação simples, baseado na comparação de elementos adjacentes e na troca de posições quando necessário. É considerado um método de ordenação local, ou seja, não cria uma nova lista, mas realiza trocas diretamente dentro da estrutura original.

2.2 Nivel de complexidade

Sua complexidade é considerada relativamente baixa, pois se trata de um código simples, no entanto seu desempenho pode ser prejudicado para grandes conjuntos de dados, visto que o processo de comparação é repetido diversas vezes entre todos os elementos, o que o torna um método exaustivo e com desempenho inferior em termos de velocidade de processamento.

2.3 Como funciona

Para o funcionamento adequado desse algoritmo, utiliza-se um laço de repetição que se encerra apenas quando não há mais trocas após uma passagem completa pelos elementos do vetor. Dentro do laço, ocorre a comparação entre um elemento e o seu vizinho imediato (seguinte ou anterior, conforme a direção da ordenação). Caso a condição de troca seja atendida, os elementos são permutados e o contador de trocas é incrementado. O processo se repete até que o número de trocas seja igual a zero, garantindo que todos os elementos estejam devidamente ordenados.

2.4 Vantagens e desvantagens

Como uma das suas principais vantagens temos a sua simplicidade, sendo uma escolha simples para iniciantes devido ao fato de que sua complexidade é baixa e o algoritmo é fácil de entender e implementar.

No entanto com desvantagens temos como desvantagem ao ordenar grandes conjuntos de dados. Seu desempenho é quadrático no pior caso, o que significa que seu tempo de execução aumenta quadrado com o tamanho da entrada. Um método pouco utilizado por programadores experientes

2.5 Código exemplo



```

1
2
3
4
5
6  #include <locale.h>
7  #include <stdio.h>
8
9  int main() {
10     int vetor[] = {64, 34, 25, 12, 22, 11, 90};
11     int tamanho = sizeof(vetor) / sizeof(vetor[0]);
12     int i, j, temp;
13
14     printf("Vetor original:\n");
15     for (i = 0; i < tamanho; i++) {
16         printf("%d ", vetor[i]);
17     }
18
19     // Algoritmo Bubble Sort (sem função)
20     for (i = 0; i < tamanho - 1; i++) {
21         // Últimos i elementos já estão na posição correta
22         for (j = 0; j < tamanho - i - 1; j++) {
23             // Se o elemento atual for maior que o próximo, troca
24             if (vetor[j] > vetor[j + 1]) {
25                 temp = vetor[j];
26                 vetor[j] = vetor[j + 1];
27                 vetor[j + 1] = temp;
28             }
29         }
30     }
31 }
32

```

Line: 3 Col: 1 Sel: 0 Lines: 32 Length: 753 Insert

3 Método Selection Sort

3.1 O que é

O método Selection Sort também é um algoritmo de ordenação simples, apresentando certa semelhança com o Bubble Sort. Entre suas principais vantagens estão a baixa complexidade conceitual e o não uso de vetores auxiliares, o que reduz o consumo de memória. Por esse motivo, o Selection Sort é considerado eficiente para a ordenação de vetores pequenos. Entretanto, para vetores de grande dimensão, o algoritmo torna-se menos eficiente, uma vez que realiza aproximadamente $(n^2-n)/2$ comparações, independentemente de o vetor estar previamente ordenado ou não.

3.2 Implementação

A implementação do Selection Sort consiste em comparar um elemento com todos os demais, identificando o menor (ou maior, dependendo da ordem desejada) e posicionando-o corretamente. Em uma ordenação crescente, por exemplo, o primeiro elemento é comparado com o segundo, o segundo com o terceiro e assim sucessivamente, até que o menor valor seja encontrado e colocado na primeira posição. O processo é então repetido com o segundo elemento, ignorando o primeiro, que já está na posição correta.

3.3 Eficiência

Este método não se destaca pela eficiência prática, porém, quando utilizado em vetores com baixa quantidade de elementos se torna consideravelmente rápido, mas em geral é mais utilizado de forma didática assim como o Bubble Sort, em base ele é um tipo de método que não necessita da criação de uma variável auxiliar, tornando o uso de memória um pouco menor.

3.4 Comparação

3.5 Este método se torna superior ao Bubble Sort no quesito de processamento, pois realiza um número menor de trocas. Enquanto o Bubble Sort efetua múltiplas trocas em cada sequência de comparações, o Selection Sort realiza apenas uma troca por interação o que reduz o tempo de execução e melhora o desempenho em vetores pequenos, em contraponto o Selection Sort não é tão estável quanto o Bubble Sort, em casos em que há um número repetido o método não consegue definir qual o maior de forma clara realizando trocas desnecessárias. Para grandes entradas em geral é mais recomendado o quick sort ou o Merge So

3.6 Código exemplo



```

1
2
3
4
5 #include<locale.h>
6 #include <stdio.h>
7
8 int main() {
9     int vetor[] = {29, 10, 14, 37, 14};
10    int tamanho = sizeof(vetor) / sizeof(vetor[0]);
11    int i, j, minIndex, temp;
12
13    printf("Vetor original:\n");
14    for (i = 0; i < tamanho; i++) {
15        printf("%d ", vetor[i]);
16    }
17
18    for (i = 0; i < tamanho - 1; i++) {
19        minIndex = i;
20
21        for (j = i + 1; j < tamanho; j++) {
22            if (vetor[j] < vetor[minIndex]) {
23                minIndex = j;
24            }
25        }
26
27        if (minIndex != i) {
28            temp = vetor[i];
29            vetor[i] = vetor[minIndex];
30            vetor[minIndex] = temp;
31        }
32    }
33
34    printf("\n\nVetor ordenado (Selection Sort):\n");
35    for (i = 0; i < tamanho; i++) {
36        printf("%d ", vetor[i]);
37    }
38
39    printf("\n");
40    return 0;
41 }
42
43

```

Line: 43 Col: 1 Sel: 0 Lines: 43 Length: 856 Insert

4 Método Insertion Sort

4.1 O que é

Selection Sort também é um método de ordenação, e é mais estável se a ordem relativa dos itens iguais não se altera durante a ordenação. Ele possui um funcionamento simples que consiste na comparação de um elemento com seus vizinhos, e apesar de ser parecido, sua grande diferença que o separa do Bubble sort, é que este método realiza apenas uma varredura, ou seja ele lê o vetor apenas uma vez e realiza as trocas entre os elementos a partir desta comparação

4.2 Como funciona

O método Selection Sort em base funciona da seguinte forma, inicia-se um laço de repetição que lê o primeiro elemento e o compara com o próximo, caso o elemento seguinte possua um valor menor que o dele, ele realiza uma troca de posição colocando o elemento de menor número na casa mais a esquerda, após a troca ele inicia um laço de repetição que compara o elemento trocado com o elemento anterior e caso ele seja menor haverá mais uma troca, o laço de repetição se encerra apenas quando este elemento não for menor ao elemento anterior, ou chegue ao início do vetor, em ambos os casos a “troca” se torna falsa e as comparações retornam a ordem crescente, comparando o elemento ao próximo até o fim do vetor, quando o mesmo se encerra ele sai do laço com o vetor já ordenado.

4.3 Vantagens e Desvantagens

A maior vantagem do Selection Sort é o seu baixo consumo de memória, fora que possui uma ótima estabilidade, a sua leitura única do vetor impede ciclos enormes de repetição, tornando o processo menos exaustivo e melhorando o processamento e a velocidade, fora que este método tem uma ótima dinâmica permitindo conferir os elementos novos enquanto eles chegam.

Em contraponto como desvantagem temos que o desempenho continua ruim ao tratar de grandes vetores, sua complexidade de tempo no pior dos casos é quadrática, o que o torna lento quando necessário grande comparações, e apesar de fazer apenas uma varredura ele realiza muitas trocas.

4.4 Comparações

Este método é mais rápido que o Bubble Sort devido ao fato de realizar menos leituras, no entanto a quantidade de trocas continua igual, e também é mais rápido eliminando o problema do Selection Sort de processar o mesmo vetor mesmo quando ele já está ordenado

4.5 Código de exemplo



```

1
2
3
4
5
6 #include <stdio.h>
7
8 int main() {
9     int vetor[] = {12, 11, 13, 5, 6};
10    int tamanho = sizeof(vetor) / sizeof(vetor[0]);
11    int i, j, chave;
12
13    printf("Vetor original:\n");
14    for (i = 0; i < tamanho; i++) {
15        printf("%d ", vetor[i]);
16    }
17
18    // Algoritmo Insertion Sort
19    for (i = 1; i < tamanho; i++) {
20        chave = vetor[i];
21        j = i - 1;
22
23        while (j >= 0 && vetor[j] > chave) {
24            vetor[j + 1] = vetor[j];
25            j = j - 1;
26        }
27        vetor[j + 1] = chave;
28    }
29
30    printf("\n\nVetor ordenado (Insertion Sort):\n");
31    for (i = 0; i < tamanho; i++) {
32        printf("%d ", vetor[i]);
33    }
34
35    printf("\n");
36    return 0;
37 }
38
39

```

Compiler Resources Compile Log Debug Find Results

Line: 39 Col: 1 Sel: 0 Lines: 39 Length: 730 Insert

5 Método Merge Sort

5.1 O que é

Merge Sort é um método de ordenação que compartilha da ideia “dividir pra conquistar”, ele funciona dividindo recursivamente o vetor de entrada em duas metade e as classifica para finalmente fundilas novamente, por ter que armazenar a metade do vetor para que as duas partes sejam lidas e ordenadas é necessário criar novos vetores, o que acaba por utilizar um pouco mais da memória, de contraponto se afirma que a velocidade deste método tem uma evolução gigantesca em comparação as demais.

5.2 Como funciona

O método Merge Sort, recebe o valor inteiro do vetor por hora desordenado, e separa ele em outros dois vetores menores dividindo-o no meio, caso o mesmo ainda esteja muito grande, ele o faz de novo, desta forma “reduzindo” a quantidade de elementos a serem ordenados e trocados, em seguida combina dois vetores pré ordenados em outro também, e retornando sucessivamente ao vetor completo, só que agora de forma ordenada.

5.3 Vantagens e Desvantagens

O Merge Sort é um dos métodos mais rápidos para ordenação de vetores, e também é ideal para vetores com grande quantidade de elementos, as subdivisões do mesmo vetor para a comparação de elementos menores de forma individual, torna o processamento constante e rápido independente do tamanho do vetor. No entanto tem-se por desvantagem quando relacionado a vetores de menor quantidade de dados, o Merge Sort se mostra menos eficiente a outras metodologias de ordenação, outra desvantagem é o consumo de memória ser maior devido ao fato de sub-dividir os vetores e ter de armazená-los em novos vetores, e no processo inverso de concatenação o problema continua.

5.4 Comparação

O Merge Sort se mostra muito eficiente como forma de solução a vetores de grandes quantidades, sendo superior a outros métodos em questão de velocidade e eficiência nesses casos, no entanto em casos de vetores menores, métodos como o Bubble Sort ou Selection Sort se mostram mais eficazes. O uso de memória exacerbado também se torna um ponto negativo em comparação as demais metodologias, fora sua complexidade o que o torna realmente um método específico para programadores com um pouco mais de experiência e relacionados a vetores com grandes quantidades de dados.

5.5 Código Exemplo



```

1
2
3
4 #include <stdio.h>
5
6 int main() {
7     int n, i, j, meio, inicio, fim, k;
8
9     printf("Digite o tamanho do vetor: ");
10    scanf("%d", &n);
11
12    int vet[n], aux[n];
13
14    printf("Digite os elementos do vetor:\n");
15    for(i = 0; i < n; i++) {
16        scanf("%d", &vet[i]);
17    }
18
19    for(meio = 1; meio < n; meio *= 2) {
20        for(inicio = 0; inicio < n - meio; inicio += 2 * meio) {
21            fim = inicio + 2 * meio;
22            if(fim > n)
23                fim = n;
24            int p1 = inicio;
25            int p2 = inicio + meio;
26            k = inicio;
27
28            while(p1 < inicio + meio && p2 < fim) {
29                if(vet[p1] <= vet[p2]) {
30                    aux[k++] = vet[p1++];
31                } else {
32                    aux[k++] = vet[p2++];
33                }
34            }
35
36            while(p1 < inicio + meio) {
37                aux[k++] = vet[p1++];
38            }
39
40            while(p2 < fim) {
41                aux[k++] = vet[p2++];
42            }
43
44            for(i = inicio; i < fim; i++) {
45                vet[i] = aux[i];
46            }
47        }
48    }
49
50    printf("\nVetor ordenado:\n");
51    for(i = 0; i < n; i++) {
52        printf("%d ", vet[i]);
53    }
54    printf("\n");
55
56    return 0;
57 }
58

```

Line: 58 Col: 1 Sel: 0 Lines: 58 Length: 1275 Insert

6 Método Quick Sort

6.1 O que é

Quick Sort também é um método de ordenação que compartilha da ideia “dividir pra conquistar”, e apesar de ser da mesma classe de complexidade do Merge Sort, o Quick Sort na prática é o mais veloz deles, pois suas constantes são menores. Ele é um método de ordenação que escolhe um pivô e separa o vetor classificado baseando-se na divisão de vetores menores, este código possui uma chamada recursiva, e esta recursão só se encerra quando o vetor já estiver ordenado.

6.2 Como funciona

O funcionamento do Quick Sort baseia-se em uma rotina fundamental cujo nome é particionamento. Particionar significa escolher um número qualquer presente no vetor, chamado de pivot, e colocá-lo em uma posição tal que todos os elementos à esquerda são menores ou iguais e todos os elementos à direita são maiores. Basicamente ele subdivide o vetor a partir de um ponto central que é usado como parametro geral para os demais elementos.

6.3 Vantagens e Desvantagens

Sua atuação local permite com que o uso de memória seja bem menor, fora que é uma das metodologias mais rápidas na prática para grandes conjuntos de dados, com uma complexidade de tempo médio, possui um laço interno simples, no entanto há algumas desvantagens como por exemplo, caso a escolha do pivô seja inconsistente o despenho pode ser significativamente pior que outros algoritmos, há conflitos em casos de listas já ordenadas ou inversamente ordenadas o que torna este método instável em seu pior caso que pode ser quadrático.

6.4 Comparação

A menor quantidade de uso de memória torna esta metodologia superior ao Merge sort, e continua a ter um desempenho se não tanto quanto, melhor do que o Merge Sort em casos de vetores com grande quantidades de dados possuindo vários elementos, provando-se superior ao Bubble Sort e ao Selection Sort neste quesito, no entanto sua falta de estabilidade o impede de ser uma metodologia muito utilizada, devido as atribuições específicas do pivô geralmente é um algoritmo mais complexo, e exercido por programadores com mais experiência principalmente por conta do uso da recursividade, não muito recomendado para iniciantes

6.5 Código exemplo



```

1
2
3
4 #include <stdio.h>
5
6 int main() {
7     int n;
8     printf("Quantidade de elementos: ");
9     if (scanf("%d", &n) != 1 || n <= 0) {
10         printf("Entrada inválida.\n");
11         return 1;
12     }
13
14     int arr[n];
15     printf("Digite %d elementos (separados por espaço ou linha):\n", n);
16     for (int i = 0; i < n; i++) {
17         scanf("%d", &arr[i]);
18     }
19
20     int stack_low[n];
21     int stack_high[n];
22     int top = -1;
23
24     top++;
25     stack_low[top] = 0;
26     stack_high[top] = n - 1;
27
28     while (top >= 0) {
29         int low = stack_low[top];
30         int high = stack_high[top];
31         top--;
32
33         int pivot = arr[high];
34         int i = low - 1;
35         for (int j = low; j <= high - 1; j++) {
36             if (arr[j] <= pivot) {
37                 i++;
38             }
39         }
40     }
41 }
42

```

Line: 42 Col: 1 Sel: 0 Lines: 42 Length: 809 Insert

7 Referências

1-<https://elemarjr.com/clube-de-estudos/artigos/o-que-e-e-como-funciona-o-bubblesort>

2- https://www-w3schools-com.translate.goog/dsa/dsa_algo_bubblesort.php?_x_tr_sl=en&_x_tr_tl=pt&_x_tr_hl=pt&_x_tr_pto=tc

3- <https://youtu.be/8RkZoBZNNgl?si=ZVK9mRYTmFFiWNJb>

4-<https://pt.stackoverflow.com/questions/165064/diferença-entre-os-métodos-de-ordenação-selection-sort-insertion-sort-e-bubble>

5- <https://youtu.be/vHxtP9BC-AA?si=vw5My0Od7yHXyVij>

6- https://youtu.be/N8iBOuoqtJE?si=CLRHVj8b5_rm1Ji-

7- https://www-geeksforgeeks-org.translate.goog/dsa/selection-sort-algorithm-2/?_x_tr_sl=en&_x_tr_tl=pt&_x_tr_hl=pt&_x_tr_pto=tc

8- https://www-w3schools-com.translate.goog/dsa/dsa_algo_selectionsort.php?_x_tr_sl=en&_x_tr_tl=pt&_x_tr_hl=pt&_x_tr_pto=tc

9- https://www-programiz-com.translate.goog/dsa/selection-sort?_x_tr_sl=en&_x_tr_tl=pt&_x_tr_hl=pt&_x_tr_pto=tc

10- <https://www.dio.me/articles/trabalho-academico-algoritmo-de-ordenacao-selection-sort-92aef5fc1119>

11- <https://youtu.be/ECdLOLaIVx8>

12- <https://www.dio.me/articles/insertion-sort-estrutura-de-dados>

13- https://ww2.inf.ufg.br/~hebert/disc/aed1/AED1_04_ordenacao1

14- <https://www.geeksforgeeks.org/dsa/merge-sort>

15- <https://guilherme-rmendes95.medium.com/algoritmos-merge-sort-ef12dadeba2a>

16- <https://joaoarthurbm.github.io/eda/posts/merge-sort>

17- https://www-w3schools-com.translate.goog/dsa/dsa_algo_quicksort.php?_x_tr_sl=en&_x_tr_tl=pt&_x_tr_hl=pt&_x_tr_pto=tc

18- <https://youtu.be/wU7Q8Z51MUI>

Vídeo: <https://youtu.be/yBZV9ftHIU8>